COMP 3920 Database Systems 1

MySQL
Scheduled Events

Introduction:
The goal of this document is to serve as a walkthrough working with Scheduled Events, in MySQL 8.

We will use the database from a previous week called **classicmodels**.

Scheduled Events are blocks of code and queries (like stored procedures) that can be run as one-time future tasks or on regular intervals. Good examples of scheduled events are cleanup and maintenance tasks. Perhaps you want to delete old unused data, perhaps you want to summarize large tables for easy access later.

Step 1:
Login to MySQL as user `root`.
Switch the default database to `classicmodels`.

Step 2:
Let's ensure that the Event Scheduler is running.

Run the following command:
```
SHOW VARIABLES LIKE "EVENT%";
```

If your event scheduler is running you should see `event_scheduler = ON`.

```
mysql> show variables like "event%";
+-----------------+-------+
| Variable_name   | Value |
+-----------------+-------+
| event_scheduler | ON    |
+-----------------+-------+
1 row in set (0.00 sec)

mysql>
```

Another way to check that the event scheduler is running is to see it in the processlist:
```
SHOW PROCESSLIST;
```

Typically the first entry in the results will be for the event_scheduler process:

```
mysql> SHOW PROCESSLIST;
+----+----------------+----------------+---------------+---------+-------+------------------------+------------------+
| Id | User           | Host           | db            | Command | Time  | State                  | Info             |
+----+----------------+----------------+---------------+---------+-------+------------------------+------------------+
|  4 | event_scheduler | localhost      | NULL          | Daemon  | 57500 | Waiting on empty queue | NULL             |
```

We'll create a new entry in our `classicmodels.systemvariables` table:

```
select * from classicmodels.systemvariables;
```

Result Grid | Filter Rows: | Edit:

systemvariables_id	keyname	thevalue
1	nextProductId	5003
2	employeeCount	24
3	eventCount	0

Create the new row entry with:
`systemvariables_id = 3,`
`keyname = "eventCount"` and
`thevalue = 0`

Step 3:

We are going to create a demo event to see if the event scheduler works.

Execute the following statements:

```
delimiter |
CREATE EVENT eventCount
    ON SCHEDULE
      EVERY 5 SECOND
    DO
      BEGIN
          UPDATE systemvariables
          SET thevalue = thevalue + 1
          WHERE keyname = 'eventCount';
    END |
delimiter ;
```

This will create a scheduled event called eventCount which will increment our new system variable 'eventCount' every 5 seconds.

Submit a screenshot of the query:

```
SELECT classicmodels.systemvariables.*, NOW()
FROM classicmodels.systemvariables;
```

01_EventCount_Now.jpg

Submit

```
select classicmodels.systemvariables.*, NOW()
from classicmodels.systemvariables;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

systemvariables_id	keyname	thevalue	NOW()
1	nextProductId	5003	
2	employeeCount	24	
3	eventCount	2	

Submit a screenshot of the query:

```
SELECT classicmodels.systemvariables.*, NOW()
FROM classicmodels.systemvariables;
```

02_EventCount_Later.jpg

Submit

```
select classicmodels.systemvariables.*, NOW()
from classicmodels.systemvariables;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

systemvariables_id	keyname	thevalue	NOW()
1	nextProductId	5003	
2	employeeCount	24	
3	eventCount	11	

Now that we have proven that scheduled events work on our database, let's disable this event so it doesn't keep updating our systemvariables table every 5 seconds.

Run:

```
ALTER EVENT eventCount DISABLE;
```

Run the following query to show a list of events created in the current database:

```
SHOW EVENTS \G
```

Submit a screenshot of the query (make sure to include the entire output):

Submit

03_Show_Events.jpg

```
mysql> show events \G
*************************** 1. row ***************************
                  Db: classicmodels
                Name: eventCount
             Definer: root@localhost
           Time zone: [redacted]
                Type: [redacted]
          Execute at: [redacted]
      Interval value: [redacted]
      Interval field: [redacted]
              Starts: [redacted]
                Ends: [redacted]
              Status: [redacted]
          Originator: [redacted]
character_set_client: [redacted]
collation_connection: [redacted]
  Database Collation: [redacted]
1 row in set (0.00 sec)

mysql>
```